NFMap: A Comprehensive Technical and Pedagogical Guide

1. Introduction

Welcome! If you are reading this, you are looking to master **NFMap** (Normalizing Flow Mapping). This guide is designed to be highly pedagogical—starting from simple analogies suitable for a first-year machine learning student, and gradually diving into the deepest mathematical and algorithmic details of the method. By the end, you will have the knowledge needed to explain, improve, or publish upon this method.

At its core, NFMap is a dimensionality reduction and clustering algorithm that marries two powerful worlds in machine learning: 1. **Normalizing Flows (Generative Models):** They provide exact probability density estimation and invertible mappings between a simple latent space and a complex data space. *(Concept popularized by Rezende & Mohamed, 2015)* 2. **Topological Clustering (Graph-based):** Methods that use k-Nearest Neighbors (k-NN) graphs to provide powerful forces that pull similar points together into tight, distinct clusters. *(Core mechanics pioneered by t-SNE [van der Maaten & Hinton, 2008] and UMAP [McInnes et al., 2018])*

Combining these two forces sounds like a match made in heaven, but mathematically, they strongly clash. NFMap's primary innovation is solving the **"Topological Deadlock"** that occurs when these two forces collide.

---

2. What are Normalizing Flows?

The "Squishy Box" Analogy (Change of Variables)

Imagine you have a 1-liter glass box containing exactly 10 gas particles. The "density" of this box is 10 particles per liter. Now, imagine you squish the box down so it only has a volume of 0.5 liters, but no particles escape. The density has just doubled to 20 particles per liter!

A **Normalizing Flow** treats data space like an infinitely stretchy, squishy rubber space. It maps messy, complex data (like images) into a neat, perfectly symmetrical Gaussian blob (the latent space). Because the mapping is continuous and invertible (a **bijection**), we can track exactly how much the "volume" was squished or stretched at every single point.

If we have a data point x and a latent point $z = f^{-1}(x)$, the probability density of x is perfectly given by the **Change of Variables Formula**:

$$\log p_X(x) = \log p_Z(z) + \log \left| \det \left(\frac{\partial z}{\partial x} \right) \right|$$

Let's break this down intuitively: - $\log p_Z(z)$: This is simply the probability of the point in the neat Gaussian grid. - $\det\left(\frac{\partial z}{\partial x}\right)$: This is the **Jacobian Determinant**. This is literally just the "volume scaling factor" from the squishy box analogy! It tells us exactly how much the flow stretched or squashed the space around x when mapping it to z. If the flow squashed the volume by half, the density doubles.

Because the mapping is bijective, if you learn how to warp the messy data into a neat grid, you can run the process in *reverse* to generate brand new data points that perfectly mimic your original data.

The Algorithm: RealNVP (Affine Coupling)

Calculating that Jacobian determinant for thousands of dimensions is computationally impossible for a standard neural network. Normalizing Flows like **RealNVP** *(Dinh et al., 2016)* solve this by splitting the input dimensions in half. They use the first half (x_1) to predict how to scale and shift the second half (x_2).

Mathematically, a single RealNVP block does this:

$$y_1 = x_1$$

$$y_2 = x_2 \odot \exp(s(x_1)) + t(x_1)$$

Because y_1 doesn't change, the Jacobian matrix becomes "triangular". The determinant of a triangular matrix is just the product of its diagonal. Therefore, the complex volume scaling factor $\log|\det J|$ simplifies to just the sum of the scale outputs: $\sum s(x_1)$. This makes it lightning fast!

Here is dummy code for a RealNVP block:

```
def forward(x):
    # Split input in half
    x1, x2 = split(x)

    # Neural network predicts scale (s) and translation (t) using ONLY x1
    s, t = neural_net(x1)

    # Apply transformation to x2
    y1 = x1
    y2 = x2 * exp(s) + t

    # The volume change is trivially the sum of 's'
    log_det = sum(s)

    return concat(y1, y2), log_det
```

---

3. What Does NFMap Do?

Methods that rely purely on k-NN graphs are fantastic at taking high-dimensional data and finding tight 2D clusters. However, they are point-to-point mappings. You cannot easily ask them: “What is the exact probability density of this new point?” or “Generate a new point from this empty spot on the 2D plane.”

NFMap fixes this by using a Normalizing Flow to perform the dimensionality reduction.

It maps a high-dimensional data space (e.g., a 64-dimensional space) into a latent space of the *same* dimensionality. However, it forces the data to lie on a **2D manifold** within that high-dimensional latent space. The first 2 dimensions become our clustering plane, and the remaining dimensions (e.g., the other 62) are squashed to zero. *(This manifold flow approach builds upon work like Brehmer & Cranmer, 2020)*.

(Note: “64 dimensions” is used purely as an example here. In the actual NFMap code, the dimensionality of the flow is dynamic. By default, it performs a PCA [Pearson, 1901] projection up to a maximum of 128 dimensions, and the flow operates on `n_flow_components` *dimensions, which can be explicitly set by the user or dynamically chosen to explain 95% of the variance.)*

---

4. The Topological Deadlock (The Great Challenge)

When first building NFMap, attempting to combine the exact density of Normalizing Flows with k-NN graph clustering resulted in catastrophic failure. The model froze and refused to learn, producing a “locked” embedding that looked like a basic PCA scatterplot.

We hit a **Topological Deadlock** caused by three competing mathematical forces:

1. **Volume Preservation (The Flow):** Normalizing flows must preserve the total multi-dimensional volume of the space. To tear a single blob of data into distinct, separated clusters, the flow must drastically stretch the empty space between those clusters.
2. **Manifold Compression ($\sigma_\perp$):** To force the flow to act as a 2D dimensionality reducer, we applied a strong loss that violently squashed the non-embedding dimensions to exactly 0.
3. **Graph Topology (k-NN Loss):** The k-NN graph loss attempts to pull the 2 embedding dimensions into tight, distinct clusters.

The Deadlock: Because the manifold loss violently squashed the perpendicular dimensions to zero, the volume of the space was plummeting. To compensate and preserve volume (as the bijection constraint mathematically demands), the flow was forced to stretch the remaining 2 dimensions out to infinity. However, the k-NN graph loss was simultaneously trying to pull those same 2 dimensions tightly together. These forces perfectly canceled each other out. The gradients died, and the network was physically unable to move.

---

5. NFMap's Innovations (How We Broke the Deadlock)

To break the deadlock, NFMap introduces core architectural innovations.

Innovation 1: Orthogonal Relaxation (The "Garbage Bin" Hack)

We intentionally relaxed the manifold constraint. This relies entirely on the $\sigma_\perp$ parameter.

- **What are the "Orthogonal" dimensions?** If we are mapping 64-dimensional data, our latent space is also 64-dimensional. We choose the first 2 dimensions (z_1, z_2) to be our clustering plane (the "parallel" dimensions). The remaining 62 dimensions ($z_3 ... z_{64}$) are "orthogonal" (perpendicular) to that 2D plane. *(Note: They are not colinear! Just like how the Z-axis in 3D space points straight up from an X-Y floor, in 64D space, there are 62 distinct, mutually perpendicular axes branching off the 2D plane).*
- **Why do they need to be close to zero? (The Generative Catch)** Because NFMap is a generative model! If you want to generate a brand new data point, you select a 2D point on your clustering plane, say $(z_1 = 1.5, z_2 = -0.5)$. But the Normalizing Flow requires a full 64-dimensional vector to run in reverse. How do you invent the other 62 numbers? The answer: You pad them with exactly zeros: $(1.5, -0.5, 0, 0, ..., 0)$. If the neural network was never forced during training to map the *true* data near zero in those 62 dimensions, passing that zero-padded vector backward through the flow would generate absolute garbage. The Manifold Loss ($L_{manifold}$) forces the flow to squash the true data manifold so that it intersects the $z_{perp} = 0$ plane. This ensures that zero-padding actually generates realistic data!
- **Wait, if it's a "garbage bin", aren't the values non-zero during training?** Yes, exactly! This is the crucial difference between *Training* and *Generation*:

  1. **During Training (Encoding):** When we pass real data backward through the flow, the resulting orthogonal dimensions (z_{perp}) are **not zero**. Because $\sigma_\perp$ is large, the network is free to let these values drift (e.g., to 2.5, -1.8, etc.) to absorb the volume distortion. It uses them as a true garbage bin.
  2. **During Generation (Decoding):** When we want to invent a *new* data point, we pick a 2D point, but we don't know what the garbage values should be for a point that doesn't exist yet! So, we simply guess `0.0` for all of them.

  Because the Manifold Loss gently encouraged the true data to hover *roughly* around zero (even if it drifted a bit), guessing exactly `0.0` lands us safely inside the "thickest" part of the learned data manifold, generating a highly realistic point!
- **What is the practical difference between $z_{perp} = 0$ and $z_{perp} \neq 0$?** Think of the 2D plane ($z_{parallel}$) as capturing the "macro-features" (e.g., if analyzing images, is this a cat or a dog? If analyzing molecular embeddings, what is the core chemical scaffold or binding affinity?). Think of the garbage dimensions (z_{perp}) as capturing the "micro-

noise" (e.g., background pixel static in images, or subtle, non-essential feature variances in molecular representations).

  - If you generate using exactly $z_{perp} = 0$, you are asking the network for the "perfectly clean, idealized archetype" of the data point. It represents the pure, smooth manifold. For a molecular embedding, decoding this point would likely yield the most chemically valid, prototypical structure for that region.
  - If you generate using random $z_{perp} \neq 0$ values, you are injecting noise into the decoding process. An image would look like it has TV static or unnatural artifacts. A molecular decoder might output an invalid SMILES string or a chemically unstable structure, because you have pushed the representation "off-manifold" into a region where valid data does not exist. *(Note: If you use the exact non-zero z_{perp} values that came from encoding a specific real data point, you will perfectly reconstruct that exact data point, complete with its unique micro-variations!)*

- **What happens when $\sigma_\perp$ is small (e.g., 0.01)?** The network faces a massive penalty if the non-embedding dimensions deviate even slightly from 0.0. This creates an unyielding, rigid mathematical wall. The flow is completely locked, unable to expand volume in these dimensions, triggering the topological deadlock.
- **What happens when $\sigma_\perp$ is large (e.g., 5.0 to 10.0)?** The constraint becomes a soft recommendation. The network is essentially told: *"You should try to keep the non-embedding dimensions close to zero so generation works, but if you need to stretch them wildly to make the math work, go ahead."*

By using an optimally large $\sigma_\perp$, we provide the flow with a "garbage bin" for volume expansion. When the k-NN graph loss tears the 2D plane apart to form clusters, the flow compensates for the massive volume stretching by dumping the distortion into the orthogonal dimensions. Because we don't care about the visualization of those extra dimensions, this completely bypasses the deadlock. We get tight 2D clusters, and generation still works!

Innovation 2: Learnable GMM Priors

Normally, Normalizing Flows map to a single Standard Gaussian. Since we want clusters, we map it to a Gaussian Mixture Model (GMM). However, forcing the latent space into a rigid, fixed grid of Gaussian components fights against the natural topology of the data.

In NFMap, we use `learnable_priors=True`. The centers (means) of the Gaussian components are PyTorch `nn.Parameter`s. They organically slide around the latent space during training, naturally adapting to the exact locations where the k-NN graph loss decides to form the clusters.

---

6. Mathematical Algorithm and Loss Details

Let's walk through the actual pseudo-code and mathematical formulas executed during `model.fit_transform(X)`.

Step 1: Preprocessing & k-NN Graph Construction

The data is standardized, aligned via PCA, and a k-Nearest Neighbors graph is constructed to compute transition weights (probabilities) between points.

Step 2: Forward and Inverse Flow

The neural network processes the data in batches:

```
# x is the input data batch
# Pass backward through the RealNVP flow to get latent z
z, log_det_inv = flow.inverse(x)

# Split z into the 2D plane and the orthogonal dimensions
z_parallel = z[:, :2]
z_perp = z[:, 2:]
```

Step 3: The Multi-Objective Loss Function

The total loss is a weighted sum of four distinct mathematical forces:

$$L_{total} = w_{prior}L_{prior} + L_{manifold} + w_{vol}L_{vol} + w_{graph}L_{graph}$$

1. GMM Prior Loss: Maximizes the likelihood that the 2D points belong to the sliding Gaussian blobs.

$$L_{prior} = -\log \sum_{k=1}^{K} \pi_k \mathcal{N}(z_{parallel} \mid \mu_k, \sigma_k)$$

2. Manifold Loss ($\sigma_\perp$ relaxation): The "garbage bin" penalty.

$$L_{manifold} = \frac{1}{2\sigma_\perp^2} \sum (z_{perp})^2$$

3. Volume Loss (Likelihood): Maximizes the log-determinant to ensure flow invertibility.

$$L_{vol} = -\log \left| \det \left(\frac{\partial z}{\partial x} \right) \right|$$

4. k-NN Graph Loss (The Topology):

$$L_{graph} = L_{attractive} + w_{repel}L_{repulsive}$$

- **Attractive Force:** Pulls k-NN neighbors together using a heavy-tailed distribution (prevents overcrowding).

$$L_{attractive} = \sum_{neighbors} w_{ij} \log(1 + | \, | \, z_i - z_j \, | \, |^2)$$

- **Repulsive Force:** Pushes randomly sampled non-neighbors apart.

$$L_{repulsive} = \sum_{random} \log\left(1 + \frac{1}{| \, | \, z_i - z_{random} \, | \, |^2 + \epsilon}\right)$$

Step 4: Optimization

The model optimizes this massive joint loss function using `AdamW` and early stopping to bend the high-dimensional space until the topology aligns.

---

7. Parameters and Their Meaning

To prevent formatting issues, here are the key parameters listed with their definitions and intuitions:

- **`n_components` (Default: 2):** Number of dimensions in the output clustering plane. Usually 2 or 3 for visualization.
- **`n_flow_components` (Default: None):** Number of dimensions the flow operates on. If `None`, it uses all dimensions. Can be set to a float (e.g. 0.95) to keep dimensions explaining 95% of PCA variance, speeding up large datasets.
- **`sigma_perp` (Default: 0.5): CRITICAL.** The tolerance/relaxation for the orthogonal dimensions. If clusters are failing to separate, increase this to 5.0 or 10.0 to give the flow a bigger "garbage bin". Small values (0.01) lock the flow completely. If it is too large, the generated points might be off-manifold.
- **`graph_weight` (Default: 5.0):** Strength of the k-NN attractive clustering force. Increase to 10.0 if you want tighter clusters; decrease if it overrides the generative density estimation.
- **`repulsion_weight` (Default: 1.0):** Strength of the repulsive force pushing non-neighbors apart. Increase to create more empty space between distinct clusters.
- **`prior_weight` (Default: 1.0):** Strength of the GMM clustering prior. Forces points to gather into Gaussian blobs.
- **`learnable_priors` (Default: False):** Whether GMM centers can move. Set to `True` for complex datasets where you want the clusters to organically find their own layout.
- **`use_canonical` (Default: False):** Applies Flouris & Konukoglu (2023) isometric regularization. Set to `True` if you want the 2D latent space distances to perfectly represent true geodesic distances in the data space. Adds computation time.
- **`perplexity` (Default: 30):** Size of local neighborhoods for the graph. Increase for more global structure, decrease for tighter local structure.

- `negative_sample_rate` **(Default: 5):** Number of repulsive samples per point. Controls the balance of the graph forces.

8. Summary

NFMap is a state-of-the-art bridge between generation and clustering. By utilizing **Orthogonal Relaxation**, it bypasses the topological deadlock that normally prevents Normalizing Flows from tearing manifolds apart. This results in a model that clusters as beautifully as a k-NN graph embedding, but retains the mathematical rigor, exact likelihood, and generative invertibility of a Normalizing Flow!